Analysis and Design of Algorithms

CS3230

Recitation

Week 12

Prof Lee Wee Sun

Showing NP-completeness

To prove that L is *NP-complete*

1. Find a suitable NP-complete problem L'
2. Show that $L' \leq_P L$. This shows that L is *NP-hard*
3. Show that L is in *NP*.

Reduction

To show that $L' \leq_P L$

1. Provide a transformation that transform an instance $\alpha \in L'$ to an instance $\beta \in L$ in *polynomial time*.

2. Show that the solution to $\beta \in L$ also gives the solution to $\alpha \in L'$. Usually, we show that

- A *YES* solution to $\alpha \in L'$ implies a *YES* solution to $\beta \in L$, **and**
- A *YES* solution to $\beta \in L$ implies a *YES* solution to $\alpha \in L'$

Admin

Programming assignment 3 due this Sunday midnight.

MCQ

Question 1

You are trying to come up with a polynomial time algorithm for solving problem A. After many unsuccessful and sleepness nights, you decided to try to instead prove that problem A is NP-complete. You successfully prove that A is in NP. Then you found an NP-complete problem B that is similar to A and showed that A can be reduced to B in polynomial time and hence B can be used to solve A, or in other words $A \leq_P B$. You have successfully shown that A is NP-complete.

- True
- False

Answer: B

False. To show that A is difficult, you need to find a difficult problem B, and reduce B to A, i.e. show $B \leq_P A$, i.e. A is as hard as an NP-complete problem.

Very common error: Show $A \leq_P B$ instead of $B \leq_P A$. **Don't make this mistake.**

We wish to show that problem L_2 is *NP-HARD* by transformation. We know that problem L_1 is *NP-HARD*. To show this, we reduce L_1 polynomially to L_2 (showing $L_1 \leq_p L_2$), by showing how to transform an instance I_1 of problem L_1 to an instance I_2 of problem L_2. What do we need to prove after describing the transformation?

(I) If the answer for I_1 is YES, then the answer for I_2 is YES.

(II) If the answer for I_2 is YES, then the answer for I_1 is YES.

(III) The answer for I_1 and I_2 are both YES.

Question 2

- (I) only
- (II) only
- (III) only
- (I) and (II)

Solution

Answer: D

We can show $A \Leftrightarrow B$

by showing $A \Rightarrow B$

and $B \Rightarrow A$

III is incorrect. We are trying to show equivalence, not that the answers for both statements are Yes.

Question 3

It is known that $A \leq_P B$. Which is the following is true?

I. A is polynomial time solvable implies that B is polynomial time solvable.

II. A is not polynomial time solvable implies that B is not polynomial time solvable.

III. B is polynomial time solvable implies that A is polynomial time solvable.

IV. B is not polynomial time solvable implies that A is not polynomial time solvable.

- I only
- II only
- II and III
- I and IV

Solution

Answer: C

$A \leq_P B$

II. A is not poly time solvable implies B is not poly time solvable.
Otherwise, we can solve A in poly time by reducing it to B and solving B in poly time.

III. B is poly time solvable implies A is poly time solvable.
Just reduce A to B then solve B in poly time.

$A \leq_P B$

I. A is poly time solvable implies B is poly time solvable.

IV. B is not poly time solvable implies that A is not poly time solvable.

Both are not true and are common errors. For example, all poly time solvable problems can be reduced to NP-complete problems.

Question 4

Consider the two problems:
PARTITION:
Given a set of positive integers S, can the set be partitioned into two sets of equal total sum?

HAS-ZERO:
Given a set of integers T, is the value 0 in the set?

We try to show that PARTITION $\leq_P$ HAS-ZERO using the following transformation A:
1) From the problem PARTITION, we are given S, a set of positive integers.
2) $T := \{\}$.
3) For each subset U of S, define a as the sum of the elements in U, and b as the sum of the elements in $S - U$. We add the value $a - b$ to T.
4) We use this set T for the HAS-ZERO problem.
What is wrong with this transformation?

- The transformation is correct.
- A YES solution to S does not imply a YES solution to $A(S)$
- A YES solution to $A(S)$ does not imply a YES solution to S
- The transformation does not run in polynomial time.

Solution

Answer: D

Assume S has size n.

Then the number of subsets of S is 2^n. Hence the reduction takes exponential time.

Partition is NP-complete, so we should not expect to be able to solve the problem this way.

Question 5

Who is the Master of Algorithms pictured below?

- Edsger Dijkstra
- Richard Karp
- Bob Floyd
- Alan Turing

Richard Karp

Turing Award winner

Established standard method for proving NP-completeness (Cook showed SAT, Karp showed many important problems by reduction.)

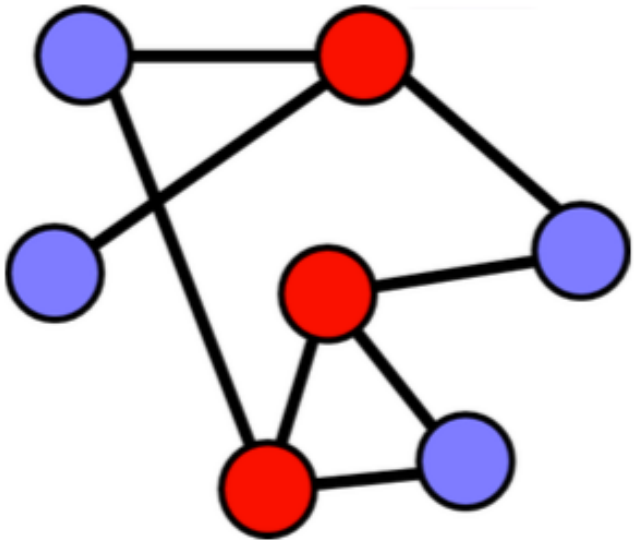
Also, Edmonds-Karp alg for max-flow, Rabin-Karp for string search, etc.

Structured Questions

Given a communication network, we would like to place programs on a small number of vertices so that we can monitor the traffic over all edges. This can be abstracted into the following decision problem:

VERTEX COVER:
Given a graph $G(V, E)$, and a subset U of the vertices V, an edge e is said to be covered by U if at least one of the endpoints of e is in U. A Vertex Cover of the graph G is a subset U of the vertices V such that all edges in the graph are covered by U.
Given a Graph G and an integer k, is there a vertex cover of size at most k?

Note: Trivially, a graph always has a vertex cover of size $|V|$.
In the diagram below, the graph has a vertex cover (the red nodes) of size 3.

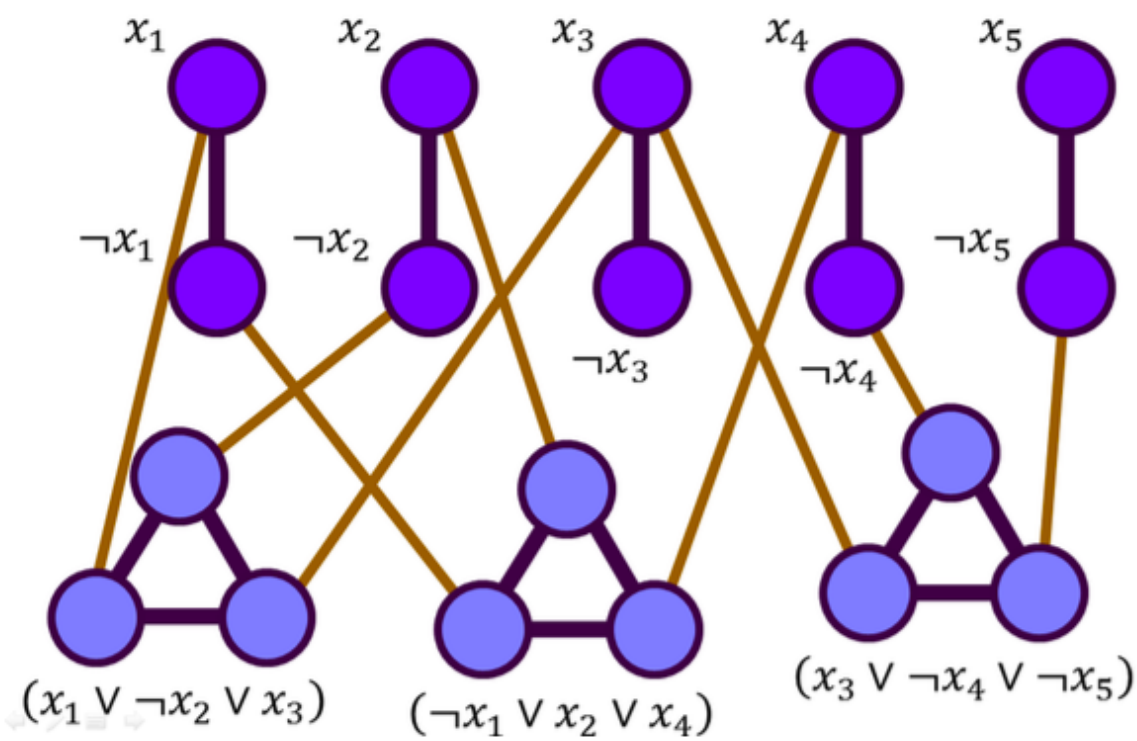


It is also well-known that VERTEX COVER is an NP-COMPLETE problem.

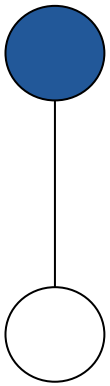
To show that VERTEX COVER is NP-HARD, we do a transformation from the 3-SAT problem. For each variable x, we create two *variable vertices* x^v and $\neg x^v$ and connect them by an edge. For each clause $(x \vee y \vee z)$ we create three *clause vertices* x^c, y^c, z^c and connect them fully with three edges. We then connect each of the clause vertice x^c to the corresponding variable vertex x^v using an edge.

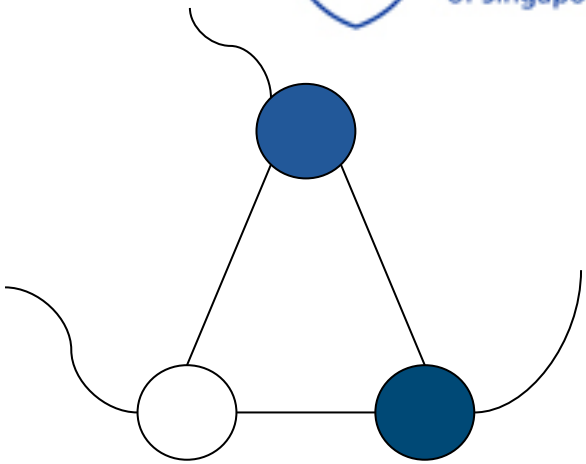
Finally, the decision problem is whether there is a cover of size at most $n + 2m$ where n is the number of variables, and m is the number of clauses.

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge (x_3 \vee \neg x_4 \vee \neg x_5)$$



Gadgets





Variable gadget (Dumbbell)

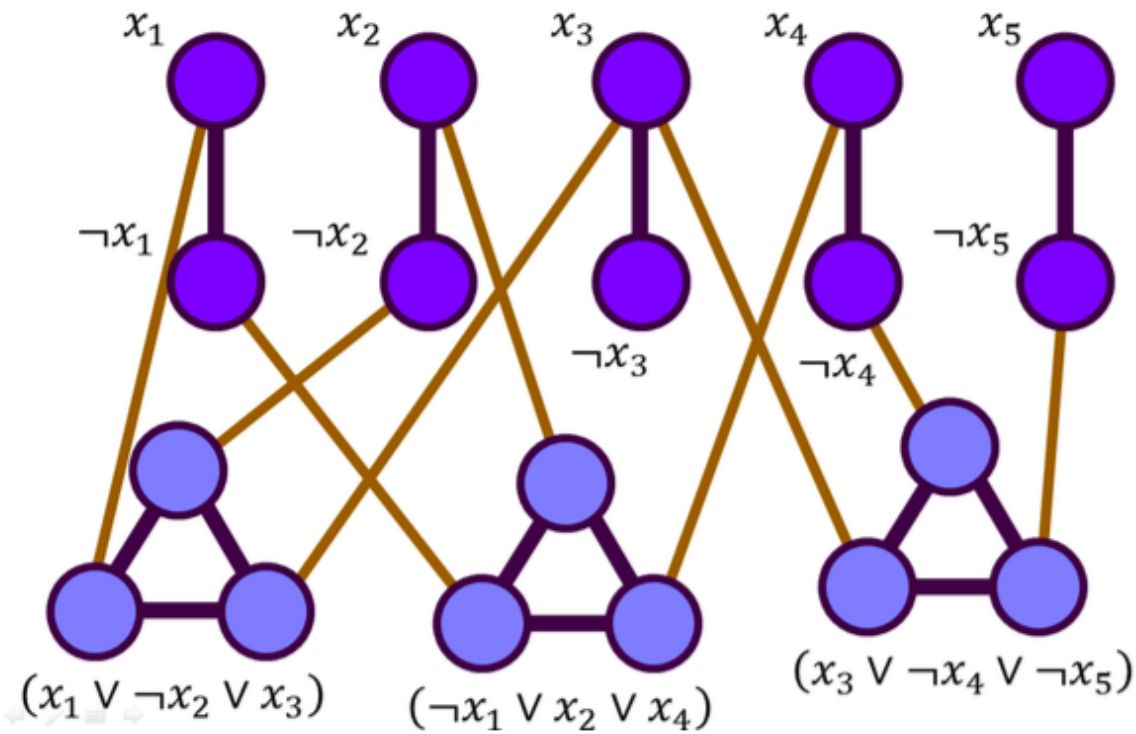
- Connects vertices labeled by variable and its negation.
- One of the two vertices must be in cover to cover dumbbell edge

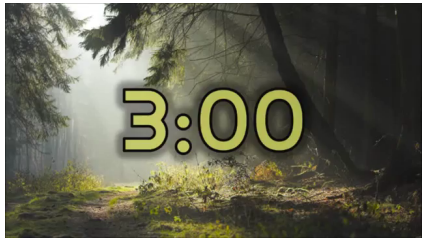
Clause gadget (Triangle)

- Fully connects vertices labeled by 3 clause literals
- Two of three vertices must be in cover to cover three triangle edges

Question 1(a)

$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge (x_3 \vee \neg x_4 \vee \neg x_5)$



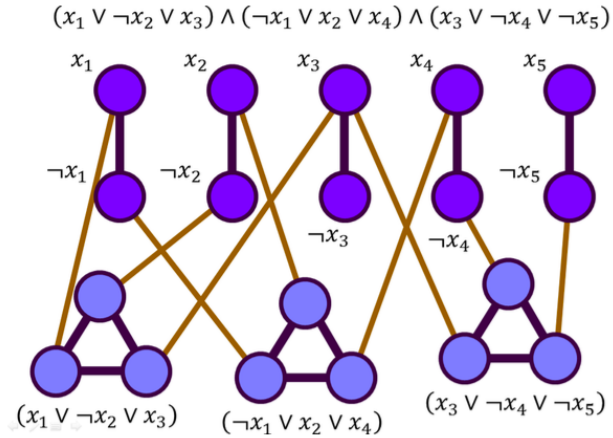


Argue that this is a polynomial time transformation. In particular, how many vertices and edges are constructed in the transformation? (n variables, m clauses)

Originally: n variables, m clauses.

Transformation:
Vertices: $2n + 3m$
Edges: $n + 6m$
The number of edges and vertices is thus polynomial on n and m.
Creating the graph thus takes polynomial time in the size of the 3-CNF formula.

Question 1(b)



Cover of size
n + 2m?

SAT soln: $x_1 = 1$, $x_2 = 0$, $x_3 = 1$, $x_4 = 1$, $x_5 = 0$

Now we prove that the transformation is correct. We prove that there is a solution to the 3-SAT problem if and only if there is a solution to the generated VERTEX COVER problem instance. First, show that if there is a solution to the 3-SAT problem, there is a solution to the generated VERTEX COVER problem instance.

Hint: For each variable x, how many of the corresponding variable vertices x^v and $\neg x^v$ must be included in the cover? For each clause $(x \vee y \vee z)$, how many of the corresponding clause vertices x^c, y^c, z^c must be included in the cover?

Assume satisfiable.

If $x_i = 1$,

add vertex x_i^v to U

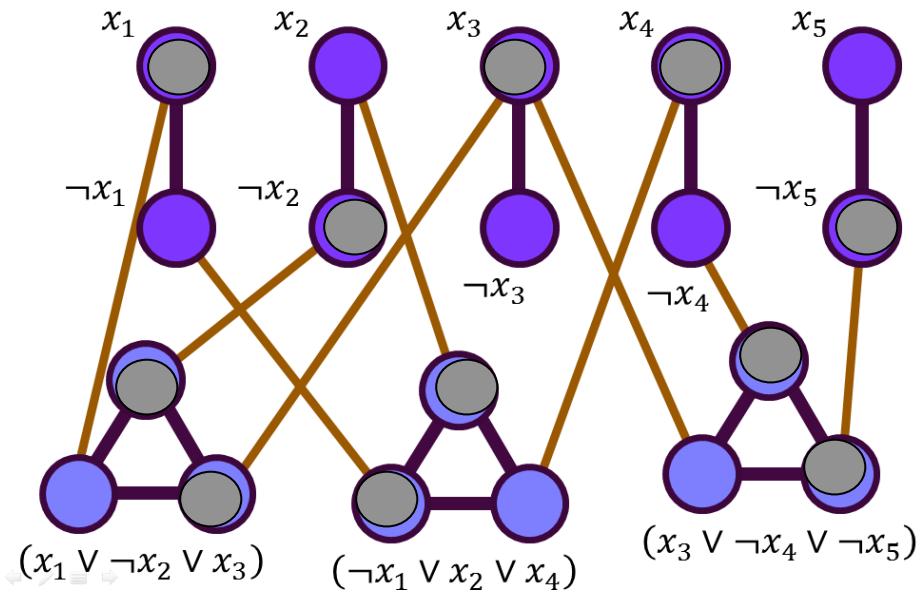
otherwise

add vertex $\neg x_i^v$ to U.

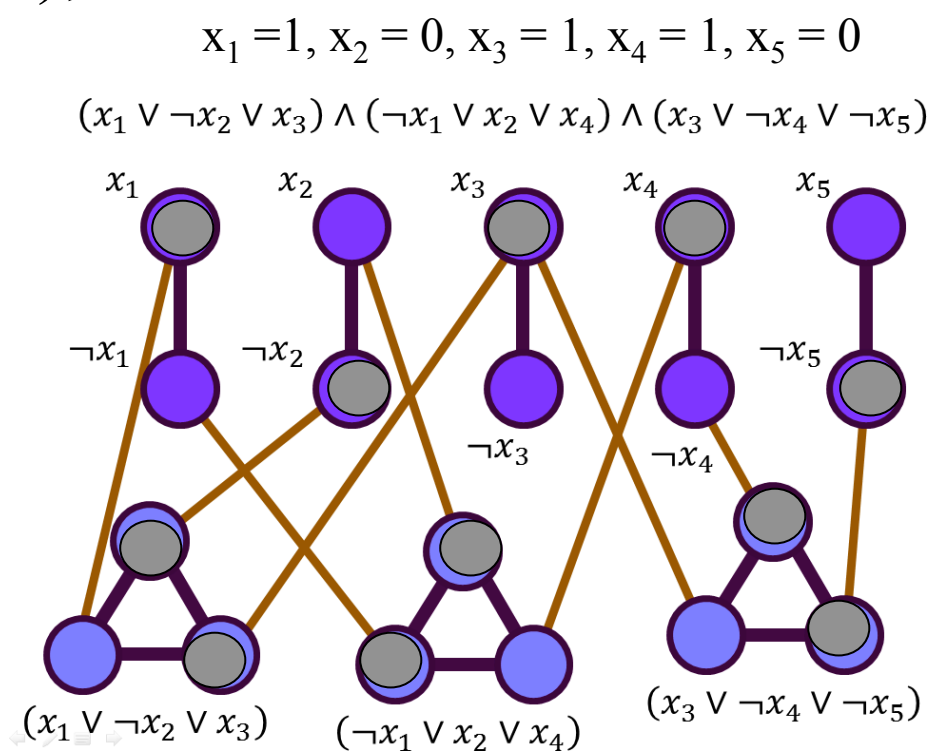
<u>Claim 1</u>: All dumbbell edges are covered

$x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 1, x_5 = 0$

$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge (x_3 \vee \neg x_4 \vee \neg x_5)$

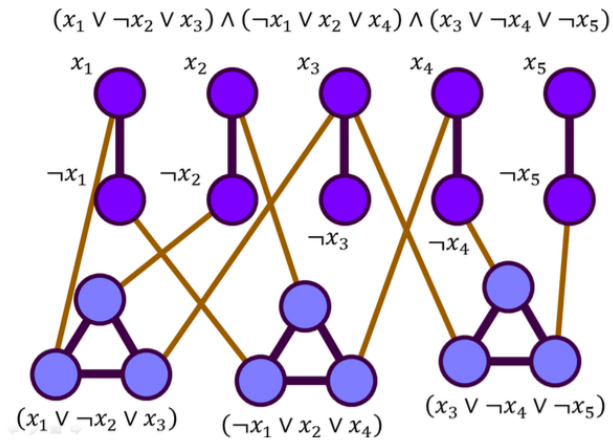


For every clause $(x \vee y \vee z)$, at least one literal, say x is set to 1 as formula satisfied.
$\rightarrow$ edge (x^v, x^c) covered as x^v in U



<u>Claim 2</u>: Adding vertices y and z in triangle to U covers all triangle edges as well as (y^v, y^c), (z^v, z^c).
Total cover size $n+2m$.

Question 1(c)

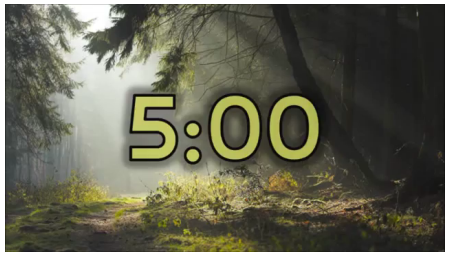


Cover of size
n + 2m?

SAT soln: $x_1 = 1$, $x_2 = 0$, $x_3 = 1$, $x_4 = 1$, $x_5 = 0$

Now show that if there is a solution to the generated VERTEX COVER problem instance, there is a solution to the 3-SAT problem.

Hint: Let *U* be the cover. Consider what happens if we remove all the edges covered by clause vertices that belong to U.

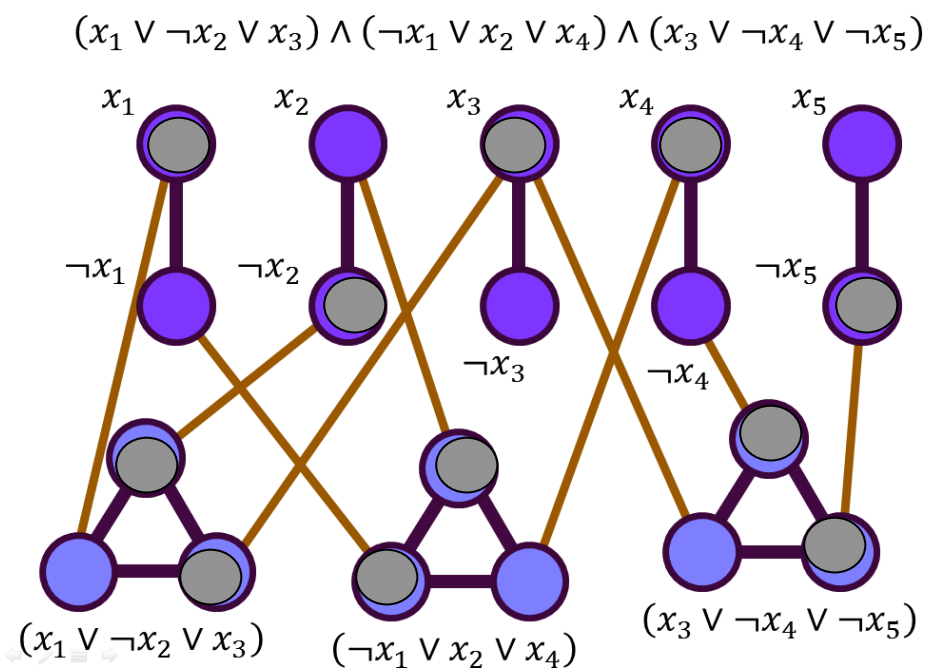


Assume cover exists.

$x_1 = 1,\ x_2 = 0,\ x_3 = 1,\ x_4 = 1,\ x_5 = 0$

For each dumbbell, exactly **one** vertex must be covered.

For each triangle, exactly **two** vertices must be covered.

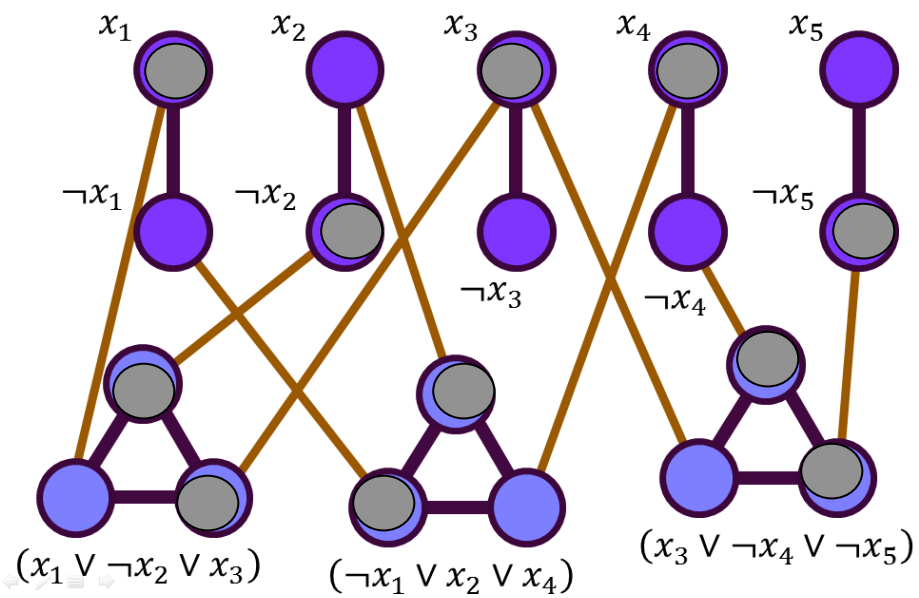


Exactly $n+2m$ in U to cover all edges.

<u>Claim:</u> Assigning literal corresponding to covered vertices in each dumbbell to 1 creates satisfying assignment to formula.

$x_1 = 1,\ x_2 = 0,\ x_3 = 1,\ x_4 = 1,\ x_5 = 0$

$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge (x_3 \vee \neg x_4 \vee \neg x_5)$

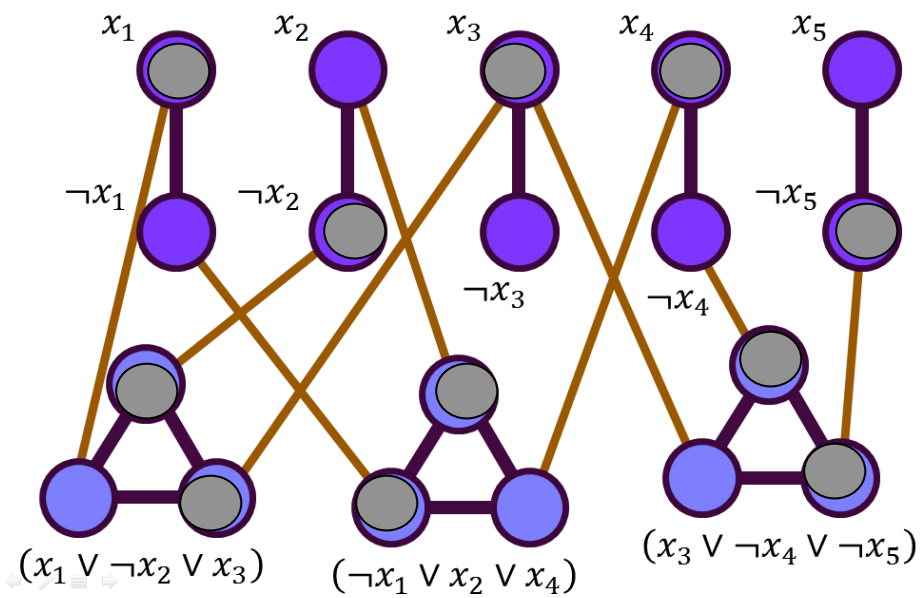


Show each clause is satisfied

1. For each triangle, delete all edges covered by the 2 vertices in U.
2. One edge (x^v, x^c) remaining for each clause $(x \vee y \vee z)$.
3. This edge (x^v, x^c) is covered by a dumbbell literal in U, hence x is assigned 1 by our assignment, and clause satisfied.

$x_1 = 1,\ x_2 = 0,\ x_3 = 1,\ x_4 = 1,\ x_5 = 0$

$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge (x_3 \vee \neg x_4 \vee \neg x_5)$



Question 1(d)

Now we want to complete the proof that VERTEX COVER is NP-COMPLETE.

Prove that VERTEX COVER is NP-COMPLETE.

Hint: What have we not shown yet?

Still need to show vertex cover is in NP.

Assume cover of size at most k exists.
The list of the vertices in the cover would work as certificate. Verify that

- number of vertices is at most k
- every edge in graph is adjacent to at least one vertex in cover.

Both steps poly time in size of graph.